

Déploiement : une API, ses tests, et la surveillance des données

Le modèle sort du notebook. Il est désormais interrogeable par une API conteneurisée, couverte par des tests, et surveillée : un indicateur mesure chaque jour si le marché ressemble encore à celui sur lequel le modèle a appris.

Dépôt DataScientest-Studio/jul26_bde_crypto Date 18 septembre 2026

9 ROUTES EXPOSÉES PAR L'API	94 TESTS AUTOMATISÉS	5 CONTENEURS DOCKER	15 JEUX SURVEILLÉS EN DÉRIVE
--------------------------------	-------------------------	------------------------	---------------------------------

01 En bref

CE QUE FAIT LE SYSTÈME AUJOURD'HUI

On lui demande une paire et un style de trading ; il répond **acheter, vendre** ou **attendre**, en s'appuyant sur les bougies stockées en base. Tout tourne dans Docker : les deux bases, l'API, et deux interfaces web pour regarder les données. Un indicateur de dérive compare en continu les données du marché à celles de l'entraînement.

Ce qui était demandé, et où le trouver

DEMANDE	RÉALISÉ	OÙ
API du modèle et des bases	OUI 9 routes, documentation interactive	api/
Tests unitaires de l'API	OUI 31 nouveaux tests, 94 au total	tests/test_api.py , tests/test_features.py
Conteneurisation	OUI image dédiée + bases, une seule commande	Dockerfile.api , docker-compose.yml
Mesure de la dérive	OUI indice PSI, en API et en ligne de commande	api/derive.py , scripts/mesurer_derive.py

Une commande suffit pour tout démarrer : `docker compose up -d` . L'API répond alors sur `localhost:8000` , avec une documentation interactive sur `/docs` où chaque route s'essaie depuis le navigateur.

02 Le machine learning, en simple

Cette section résume, sans vocabulaire technique, ce que le modèle fait et ce qu'il vaut. Le détail complet est dans le rapport de l'étape 3 et dans le notebook.

La question posée au modèle

Toutes les 15 minutes, 1 heure ou 4 heures, le marché produit une « bougie » : un résumé de ce qui s'est passé pendant cette période (prix d'ouverture, plus haut, plus bas, clôture, volume échangé). La question est : **la prochaine bougie sera-t-elle en hausse ou en baisse ?**

Pour répondre, le modèle reçoit 35 indicateurs calculés sur les bougies précédentes : écart aux moyennes mobiles, RSI, MACD, volatilité, volumes, pression acheteuse, plus l'état des échelles de temps plus lentes. Il ne voit jamais le prix en euros, seulement des rapports — c'est ce qui lui permet de travailler indifféremment sur le Bitcoin à 80 000 \$ ou le XRP à 3 \$.

Le bouton conservateur / agressif

Le modèle ne répond pas « oui » ou « non » mais donne une **probabilité** de hausse. Le style choisit à partir de quelle probabilité le bot agit réellement. En dessous, il répond *attendre*. C'est tout le bouton : un seuil.

STYLE	ACHÈTE SI LA PROBABILITÉ DÉPASSE	VEND SI ELLE DESCEND SOUS	ORDRES PAR JOUR	BONNES RÉPONSES	MARGE D'ERREUR
Agressif	0,5346	0,4315	25	0,597	0,557 – 0,636
Conservateur	0,5415	0,3921	20	0,599	0,554 – 0,642

POURQUOI DEUX SEUILS ET NON UN SEUL

Les probabilités du modèle ne sont pas symétriques : elles descendent bas du côté baisse, mais **ne dépassent jamais 0,564** du côté hausse. Avec un seuil unique mesuré depuis 0,5, le style conservateur se retrouvait au-dessus de ce maximum : il ne pouvait *structurellement* jamais acheter, et ne produisait que des ventes. Le défaut s'est vu tout de suite sur l'interface. Chaque côté a donc désormais son propre seuil, réglé pour déclencher autant de signaux. Le modèle n'était pas baissier — les hausses réelles sont à 50 % — il était simplement plus sûr de lui en annonçant une baisse.

Ces chiffres sont mesurés sur **14 831 bougies postérieures aux données d'entraînement**, collectées après coup et jamais vues par le modèle. C'est la seule mesure qui compte : elle montre qu'il n'a pas appris le passé par cœur.

Le résultat, sans l'enjoliver

LE MODÈLE A RAISON PLUS SOUVENT QU'IL N'A TORT, ET IL PERD QUAND MÊME DE L'ARGENT

Sur une bougie de 15 minutes, le prix bouge en moyenne de **0,227 %**, alors qu'un aller-retour coûte **0,2 %** de frais. Même un modèle qui ne se tromperait jamais ne gagnerait que 0,027 % par ordre. Il faudrait 94 % de bonnes réponses pour seulement couvrir les frais ; nous en sommes à 60 %.

Le backtest le confirme : sur la période récente, le style agressif perd 5,0 % et le conservateur 3,6 %, frais compris. Nous avons cherché à corriger ce point pendant deux semaines — horizons plus longs, carnet d'ordres des contrats à terme, valeurs des bougies précédentes, calibration des probabilités, prédiction de l'amplitude du mouvement. Une seule piste a apporté un gain régulier (la calibration), et aucune n'a rendu le bot rentable.

Ce n'est pas un échec de la chaîne technique, mais un résultat mesuré, conforme à la littérature : la volatilité d'un marché se prédit, sa direction non. C'est pourquoi l'API affiche un avertissement dans chacune de ses réponses.

03 L'API

ROUTE	CE QU'ELLE FAIT
<code>GET /health</code>	L'API, le modèle et les deux bases répondent-ils ? Sert aussi de sonde à Docker
<code>GET /modele</code>	D'où vient le modèle, sur quelles données, ses deux seuils, ses performances
<code>GET /paires</code>	Les paires disponibles en base
<code>GET /bougies/{paire}</code>	Les dernières bougies, lues dans PostgreSQL
<code>GET /couverture</code>	Volume et retard de collecte de chaque jeu de données
<code>POST /prediction</code>	Acheter, vendre ou attendre, selon la paire et le style
<code>GET /derive</code>	Les données récentes ressemblent-elles à celles de l'entraînement ?
<code>GET /graphique/{paire}</code>	Les dernières bougies <i>avec</i> la décision du modèle pour chacune
<code>GET /interface</code>	Une page de démonstration servie par l'API elle-même

Un appel, une réponse

```
curl -X POST http://localhost:8000/prediction \  
-H "Content-Type: application/json" \  
-d '{"symbole": "BTCUSD", "interval": "1h", "style": "conservateur"}'
```

```
{ "symbole": "BTCUSD", "interval": "1h", "style": "conservateur",  
  "bougie": "2026-08-29T10:00:00+00:00",  
  "probabilite_hausse": 0.4742, "seuil_achat": 0.5415, "seuil_vente": 0.3921,  
  "decision": "attendre", "sens": 0,  
  "avertissement": "modele experimental, non rentable frais compris" }
```

Le modèle n'est pas assez sûr de lui (0,47, entre le seuil de vente 0,39 et celui d'achat 0,54) : il s'abstient. C'est le comportement attendu la plupart du temps — en style conservateur, le bot ne se prononce que 20 fois par jour sur les cinq paires.

Une interface pour voir le modèle travailler

`http://localhost:8000/interface` affiche les bougies en chandeliers et pose une flèche à chaque fois que le modèle décide d'acheter ou de vendre. Le style se change dans la page, ce qui rend le bouton conservateur / agressif visible immédiatement : en conservateur, les flèches deviennent rares.

Deux choix rendent cette page simple à faire tourner :

- **Elle est servie par l'API elle-même**, donc à la même adresse que les routes qu'elle appelle. Aucun réglage d'origine (CORS) n'est nécessaire, et `docker compose up -d` suffit à tout lancer. Le CORS est tout de même activé, pour qu'un front-end séparé puisse appeler l'API plus tard.
- **Elle peut lire Binance en direct**. La base n'est alimentée que lorsqu'on lance la collecte, donc elle a toujours du retard ; la route `/graphique` sait aller chercher les bougies fraîches chez Binance et faire prédire le modèle dessus. La bougie en cours est écartée : sa clôture n'existe pas encore.

Le graphique est dessiné en Canvas, sans bibliothèque externe : rien à télécharger, et la page fonctionne hors ligne.

Trois choix à défendre

- **L'API lit les vues, jamais les tables**. `v_day_trading` restitue les trois pas de temps du profil, où qu'ils soient physiquement stockés. C'est exactement la raison d'être de ces vues, décidée à l'étape 2.
- **Le modèle est chargé une seule fois**, au démarrage. Le fichier pèse 99 Mo ; le recharger à chaque appel rendrait l'API inutilisable.
- **Chaque prédiction est archivée** dans la table `api_predictions`. Sans cette trace, impossible de répondre plus tard à « le modèle s'est-il mis à vendre beaucoup plus qu'avant ? », qui est le premier signe visible d'une dérive.

Sécurité — si la variable `CRYPTOBOT_API_KEY` est renseignée, toutes les routes sauf `/` et `/health` exigent un en-tête `X-API-Key`. Sans cette variable, l'API reste ouverte : acceptable en développement seulement. `/health` reste toujours accessible, sinon Docker ne pourrait plus vérifier que le service est en vie.

04 Les tests

94 tests, exécutés en 4 secondes par `pytest tests/ -q`.

FAMILLE	NOMBRE	CE QU'ELLE VÉRIFIE
API	27	Chaque route, les décisions des deux styles, les codes d'erreur (401, 404, 422, 503), la clé d'API
Variables explicatives	4	Aucun regard vers le futur, indépendance de la fenêtre de calcul et du niveau de prix
Étiquetage	~20	Barrières, absence de fuite temporelle (étape 3)
Données et configuration	~43	Nettoyage, cohérence entre le code et le schéma SQL (étapes 1 et 2)

Le principe : aucun test ne dépend des bases ni du vrai modèle

Les accès extérieurs sont remplacés par des doublures — des fausses bougies et un faux modèle qui renvoie une probabilité choisie. Trois raisons :

- les tests passent sur la machine d'un collègue, **sans Docker** ;
- ils durent quelques secondes au lieu de plusieurs minutes (le vrai modèle pèse 99 Mo) ;
- un test qui échoue désigne un bug de l'API, **pas une base éteinte**.

Exemple de test parlant : en réglant le faux modèle sur une probabilité de 0,58, on vérifie que le style agressif achète et que le conservateur s'abstient. C'est le cœur du bouton, vérifié automatiquement à chaque modification.

05 La conteneurisation

SERVICE	RÔLE	PORT
postgres	TimescaleDB : bougies nettoyées, référentiel, journaux	5432
mongo	Couche brute : réponses de l'API Binance intactes	27017
api	Le modèle et les données, exposés en HTTP	8000
pgadmin	Interface web pour PostgreSQL	5050
mongo-express	Interface web pour MongoDB	8081

Deux décisions utiles

- **Une image dédiée, plus légère.** L'API n'a besoin ni de MLflow, ni de Jupyter, ni de matplotlib : ses dépendances sont listées à part dans `requirements-api.txt`.
- **Le modèle n'est pas copié dans l'image**, il est monté depuis `./models` en lecture seule. Réentraîner le modèle ne demande donc pas de reconstruire l'image — seulement de redémarrer le conteneur.

L'API attend que les deux bases soient *saines* avant de démarrer, et expose sa propre sonde de santé : Docker redémarre le conteneur si elle cesse de répondre.

Un piège rencontré : **l'image contient le code de calcul des indicateurs.**

Après avoir corrigé un indicateur, nous avons d'abord oublié de la reconstruire, et l'API a servi pendant quelques minutes des variables différentes de celles du modèle. La commande à retenir est `docker compose up -d --build api` après toute modification de `src/`.

06 La dérivation des données

LE PROBLÈME, EN UNE PHRASE

Le modèle a appris sur une certaine période. Si le marché change — volatilité qui double, volumes qui s'effondrent, nouveau régime — les données qu'il reçoit ne ressemblent plus à celles de son apprentissage, et ses réponses deviennent douteuses. **Rien ne plante : c'est précisément le danger.**

Comment on la mesure

Pour chaque variable, on a photographié la répartition de ses valeurs pendant l'entraînement, découpée en 10 tranches de taille égale. On regarde ensuite où tombent les valeurs récentes dans ces mêmes tranches. Si la répartition est identique, chaque tranche reçoit 10 % des bougies ; si tout s'entasse d'un côté, la variable a dérivé. L'écart se résume par un indice appelé **PSI**, d'usage courant dans l'industrie :

INDICE PSI	LECTURE	ACTION
moins de 0,10	stable	rien à faire
0,10 à 0,25	dérive modérée	surveiller
plus de 0,25	dérive forte	réentraînement conseillé

Deux règles, apprises en construisant la mesure

- **On ne compare que des choses comparables.** La référence est choisie pour la paire et le pas de temps. La taille moyenne d'un trade vaut 0,005 sur le Bitcoin et 115,8 sur le XRP ; l'amplitude d'une bougie 4h vaut quatre fois celle d'une bougie 15 minutes. Comparer à une référence mélangée annonçait une dérivation massive alors que rien n'avait bougé.
- **La fenêtre se donne en jours, pas en nombre de bougies.** Trop courte, elle fait sortir en « dérivation forte » les variables lentes, simplement parce que quelques jours ne couvrent qu'une petite partie de leur distribution habituelle. Trois mois est le réglage retenu.

Mesure du 18 septembre, sur 90 jours

PAIRE	15 MIN	1 H	4 H	VARIABLE LA PLUS CONCERNÉE
BTCUSDT	STABLE	STABLE	STABLE	—
ETHUSDT	5 FORTES	1 FORTE	STABLE	taille moyenne d'un trade
BNBUSDT	5 FORTES	1 FORTE	STABLE	taille moyenne d'un trade
SOLUSDT	7 FORTES	4 FORTES	1 FORTE	taille moyenne d'un trade
XRPUSDT	7 FORTES	4 FORTES	STABLE	taille moyenne d'un trade

Le Bitcoin est stable, les autres paires non, surtout sur les pas de temps courts. Et la mesure désigne presque toujours la même coupable : la *taille moyenne d'un trade*. C'est cohérent — cette variable est la seule de notre jeu qui dépende de l'unité de la paire, donc la plus sensible à un changement de comportement du marché. Elle est la première candidate à une révision.

La mesure s'exécute aussi en ligne de commande, et chaque exécution est archivée dans docs/derive_donnees.json : ce qui compte n'est pas la valeur d'un jour, mais sa **tendance**. C'est cette commande qu'Airflow appellera quotidiennement à l'étape 5.

07 Limites et suite

LIMITE CONNUE	CONSÉQUENCE	PISTE
Le modèle n'est pas rentable	L'API est un démonstrateur, pas un outil d'investissement	Cible ignorant les mouvements trop petits pour couvrir les frais
Fichier de modèle de 99 Mo	Démarrage de l'API en quelques secondes	Réduire le nombre d'arbres, à performance quasi identique
Une variable dépend de l'unité de la paire	Première source de dérive détectée	La rapporter à sa propre moyenne récente
La base s'arrête au 29 août	La dérive se mesure sur un historique figé ; l'interface contourne le problème en lisant Binance en direct	Collecte continue, planifiée à l'étape 5

Ce que l'étape 5 apportera

- **Collecte automatique** des nouvelles bougies, chaque jour.
- **Mesure de dérive planifiée** — la commande existe déjà, il restera à la déclencher.
- **Réentraînement régulier**. C'est la piste la plus prometteuse : sur les périodes passées, le modèle reproduisait la tendance de ses données d'entraînement au lieu de suivre le marché du moment.
- **Supervision** : suivre dans le temps la répartition des décisions, déjà archivées à chaque appel.

Code `api/`, `tests/`, `Dockerfile.api` Mesures `docs/derive_donnees.json`, `docs/modele_final_direction.json`
Suivi MLflow, expérience `cryptobot_direction`